

《动手学深度学习》

PyTorch 版学习笔记

内容提炼试作

2026 年 9 月 1 日

试作说明

本 PDF 用于检验笔记的内容密度、排版和编译流程，仅提炼原书第二章“预备知识”。章内全部标题的名称、层级与顺序严格沿用 PyTorch 版原始目录。印刷目录与侧栏书签只列出章和七个主要部分，正文仍保留全部原书标题。正文按连续叙述展开：定义、成立条件、关键数学推导与结论，仅用极少量代码验证原理。

目录

试作说明	i
第二章 预备知识	1
2.1 数据操作	1
2.2 数据预处理	3
2.3 线性代数	4
2.4 微积分	7
2.5 自动微分	9
2.6 概率	11
2.7 查阅文档	13

第二章 预备知识

本章给出后续深度学习所需的共同语言：先把数据做成张量，再用线性代数写出前向计算，用微积分说明参数如何影响目标，用自动微分把求导落到程序，最后用概率描述不确定预测。正文严格保留原书标题层级，只提取定义、成立条件、关键推导和结论。

2.1 数据操作

张量是深度学习中统一的数据容器。零轴、一轴、二轴张量分别对应标量、向量和矩阵；更高阶张量可表示批量图像、视频或其他多轴数据。下面从创建形状开始，再讲运算、广播、索引，最后说明内存与互操作。

2.1.1 入门

张量的形状为各轴长度组成的元组，元素总数为各轴长度之积：

$$\text{numel}(\mathbf{X}) = \prod_{i=1}^n d_i, \quad \text{shape}(\mathbf{X}) = (d_1, \dots, d_n).$$

`reshape` 只改变张量的形状解释，不改变元素次序和总数。若其中一个轴写作 `-1`，该轴长度由元素总数和其余轴自动确定。

```
x = torch.arange(12).reshape(3, 4)
x.shape, x.numel()
```

2.1.2 运算符

形状确定之后，就可以对同形状张量做运算。加、减、乘、除和幂默认逐元素执行。若 $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ ，逐元素二元运算写为

$$\mathbf{c}_i = f(u_i, v_i), \quad i = 1, \dots, d.$$

张量连接 `cat` 沿指定轴拼接数据；比较运算返回布尔张量；`sum` 把指定范围内的元素相加。

2.1.3 广播机制

形状不完全相同也可以运算，前提是满足广播规则：从末轴向前比较，对应轴长度必须相等，或其中一个为 1。长度为 1 的轴在逻辑上复制到目标长度。例如

$$\mathbb{R}^{3 \times 1} + \mathbb{R}^{1 \times 2} \longrightarrow \mathbb{R}^{3 \times 2}.$$

2.1.4 索引和切片

运算之后需要取出或改写局部元素。索引从 0 开始，-1 表示最后一个位置；切片 $[a : b]$ 包含 a 而不包含 b 。多轴张量在各轴分别给出索引或切片，冒号表示保留该轴全部元素。

2.1.5 节省内存

反复改写张量时，赋值方式会决定是否分配新内存。 $\mathbf{Y} = \mathbf{Y} + \mathbf{X}$ 通常先创建新结果再改变变量引用；切片写回或原地运算尽量复用内存：

$$\text{id}(\mathbf{Y}) \text{ 在 } \mathbf{Y}[:] = \mathbf{Y} + \mathbf{X} \text{ 前后保持不变.}$$

参数规模很大或同一计算反复执行时，避免无必要分配可减少内存压力。

2.1.6 转换为其他 Python 对象

原地操作之所以会影响其他对象，是因为 PyTorch 张量与 NumPy 数组可共享底层内存。单元素张量可用 `item()` 转为 Python 标量；NumPy 数组可再转换回张量。

2.1.7 小结

数据操作的核心是：用形状描述张量结构，用逐元素运算和广播组合数据，用索引定位数据，并在必要时通过原地操作控制内存分配。

2.1.8 练习

原书练习集中检验三个能力：由元素数推断合法形状、说明广播后的结果形状、以及判断张量与 NumPy 对象是否共享内存。

2.2 数据预处理

原始数据往往含有缺失项和非数值字段，不能直接送入上一节的张量运算。预处理把表格变成形状明确、数值类型一致的张量，顺序是：读取、填补或编码缺失、再转换为张量。

2.2.1 读取数据集

CSV 表中通常一行对应一个样本，一列对应一个属性。读取后先划分输入属性与预测目标，再分别处理。缺失值在 pandas 中通常表示为 NaN。

2.2.2 处理缺失值

读入之后先处理缺失。数值属性可用该列均值替代缺失值。若第 j 列观测集合为 $\mathcal{I}_j$ ，均值为

$$\bar{x}_j = \frac{1}{|\mathcal{I}_j|} \sum_{i \in \mathcal{I}_j} x_{ij},$$

则缺失位置可暂以 $\bar{x}_j$ 填充。类别属性不能取算术均值，通常改为独热表示，并可把缺失本身编码为一个类别。

2.2.3 转换为张量格式

缺失处理完成后，所有字段数值化且数据类型统一，表格才可转换为张量：

$$\text{DataFrame} \longrightarrow \text{NumPy array} \longrightarrow \text{PyTorch Tensor}.$$

2.2.4 小结

预处理不是改变学习目标，而是把不完整、异质的原始字段变成模型能够计算的数值表示。

2.2.5 练习

原书练习要求统计缺失最多的列并尝试删除它，用于理解“填充”与“删除”是两类不同策略。

2.3 线性代数

张量已经可以存储数据；线性代数给出在这些对象上的运算语言。从标量、向量、矩阵逐步升到高阶张量，再由点积和矩阵乘法得到层间映射，最后用范数把误差写成可优化目标。

2.3.1 标量

标量是单个实数，记为 $x \in \mathbb{R}$ 。标量之间可进行通常的四则运算。

2.3.2 向量

把若干标量排成一行，就得到向量。本书按列向量书写：

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n.$$

2.3.2.1 长度、维度和形状

$\boldsymbol{x} \in \mathbb{R}^n$ 表示向量含 n 个分量。一维张量的形状为 $(n,)$ 。这里向量的“维度”指分量数；张量语境中的“维度”还可能指轴数。

2.3.3 矩阵

再把若干等长向量并排放在一起，就得到矩阵 $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ ，含 m 行、 n 列：

$$\boldsymbol{A} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix}.$$

转置交换行列，满足 $(\boldsymbol{A}^\top)_{ij} = a_{ji}$ 。若 $\boldsymbol{A} = \boldsymbol{A}^\top$ ，则 $\boldsymbol{A}$ 为对称矩阵。

2.3.4 张量

向量和矩阵分别是一阶、二阶张量。继续增加轴，例如图像的高度、宽度和通道，或再加批量与时间轴，就形成更高阶张量。上一节的数据操作对象正是这些张量。

2.3.5 张量算法的基本性质

对象定义之后看运算。同形状张量逐元素运算后形状不变。Hadamard 积定义为

$$\mathbf{A} \odot \mathbf{B} = [a_{ij}b_{ij}]_{m \times n},$$

它不同于矩阵乘法。张量与标量逐元素运算时，标量广播到每个位置。

2.3.6 降维

逐元素运算不改变轴数；求和则把若干轴聚合掉。对 $\mathbf{A} \in \mathbb{R}^{m \times n}$,

$$\sum_{i=1}^m \sum_{j=1}^n a_{ij}$$

得到标量；沿轴 0 求和得到长度为 n 的向量，沿轴 1 求和得到长度为 m 的向量。平均值由总和除以参与计算的元素数得到。

2.3.6.1 非降维求和

设置 `keepdim=True` 时，被聚合轴保留为长度 1，从而可借助广播做归一化。例如每行除以该行和：

$$\tilde{a}_{ij} = \frac{a_{ij}}{\sum_{k=1}^n a_{ik}}.$$

累积和则保留原轴长度，输出当前位置之前的部分和。

2.3.7 点积 (Dot Product)

降维求和把矩阵压成向量或标量；两个等长向量的点积是更结构化的求和：

$$\mathbf{x}^\top \mathbf{y} = \sum_{i=1}^d x_i y_i.$$

它可解释为加权求和；当两个向量均为单位向量时，点积等于夹角余弦。

2.3.8 矩阵-向量积

点积只作用于两个向量。把矩阵每一行都与同一向量做点积，就得到矩阵-向量积。令 $\mathbf{A} \in \mathbb{R}^{m \times n}$ 、 $\mathbf{x} \in \mathbb{R}^n$ ，则

$$\mathbf{Ax} = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{x} \\ \vdots \\ \mathbf{a}_m^\top \mathbf{x} \end{bmatrix} \in \mathbb{R}^m.$$

因此矩阵可视为从 $\mathbb{R}^n$ 到 $\mathbb{R}^m$ 的线性变换。

2.3.9 矩阵-矩阵乘法

把多个矩阵-向量积并列，就是矩阵-矩阵乘法。若 $\mathbf{A} \in \mathbb{R}^{n \times k}$ 、 $\mathbf{B} \in \mathbb{R}^{k \times m}$ ，则 $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{n \times m}$ ，其中

$$c_{ij} = \mathbf{a}_i^\top \mathbf{b}_j = \sum_{\ell=1}^k a_{i\ell} b_{\ell j}.$$

中间维度 k 必须相等；该运算可看作多个矩阵-向量积并列执行。

2.3.10 范数

前向计算得到向量或矩阵后，还需要一个标量来衡量“有多大”或“差多少”。范数把向量映射为非负标量，并满足绝对齐次性、三角不等式和正定性。 L_p 范数定义为

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}.$$

常用形式为

$$\|\mathbf{x}\|_1 = \sum_i |x_i|, \quad \|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}.$$

矩阵的 Frobenius 范数为

$$\|\mathbf{X}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n x_{ij}^2}.$$

2.3.10.1 范数和目标

训练常把“预测接近真值”写成距离最小化。例如平方误差本质上使用 L_2 范数平方：

$$\min_{\boldsymbol{\theta}} \|\mathbf{y} - f_{\boldsymbol{\theta}}(\mathbf{X})\|_2^2.$$

因此范数把几何上的距离概念转化为可优化目标。

2.3.11 关于线性代数的更多信息

本节只保留阅读后续章节所需内容。矩阵分解、低秩结构和张量分解属于进一步学习范围。

2.3.12 小结

线性代数用张量描述数据和参数，用点积及矩阵乘法描述层间映射，用范数把误差大小转成标量目标。

2.3.13 练习

原书练习要求验证转置、求和、矩阵乘法和范数性质，重点是同时检查公式与输出形状。

2.4 微积分

线性代数给出前向计算；微积分说明参数变化如何改变目标。优化研究怎样调整参数以降低训练目标；泛化研究所得模型能否在未见数据上保持效果。深度学习训练主要依赖微分而非积分。

2.4.1 导数和微分

函数 f 在 x 处的导数定义为

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h},$$

前提是该极限存在。导数同时表示瞬时变化率和曲线切线斜率。

对原书示例 $f(x) = 3x^2 - 4x$,

$$f'(x) = 6x - 4, \quad f'(1) = 2.$$

这说明在 $x = 1$ 附近，输入增加一个很小的 Δx 时，输出的一阶变化约为 $2\Delta x$ 。

常用求导法则为

$$\begin{aligned} (Cf)' &= Cf', \\ (f+g)' &= f' + g', \\ (fg)' &= f'g + fg', \\ \left(\frac{f}{g}\right)' &= \frac{f'g - fg'}{g^2}. \end{aligned}$$

2.4.2 偏导数

单变量导数只看一个自变量。对多元函数 $y = f(x_1, \dots, x_n)$ ，关于 x_i 的偏导把其余变量视为常数：

$$\frac{\partial y}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(x_1, \dots, x_i, \dots, x_n)}{h}.$$

2.4.3 梯度

把全部偏导排成一列，就得到梯度。若 $f: \mathbb{R}^n \rightarrow \mathbb{R}$,

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1} \quad \dots \quad \frac{\partial f}{\partial x_n} \right]^T.$$

原书给出的常用结果包括

$$\begin{aligned}\nabla_{\mathbf{x}} \mathbf{A}\mathbf{x} &= \mathbf{A}^T, \\ \nabla_{\mathbf{x}} \mathbf{x}^T \mathbf{A} &= \mathbf{A}, \\ \nabla_{\mathbf{x}} \mathbf{x}^T \mathbf{A}\mathbf{x} &= (\mathbf{A} + \mathbf{A}^T)\mathbf{x}, \\ \nabla_{\mathbf{x}} \|\mathbf{x}\|_2^2 &= 2\mathbf{x}, \\ \nabla_{\mathbf{X}} \|\mathbf{X}\|_F^2 &= 2\mathbf{X}.\end{aligned}$$

2.4.4 链式法则

神经网络由多层复合而成，不能对每个参数直接套用上述公式，而要把各层导数连起来。若 $y = f(u)$ 且 $u = g(x)$ ，则

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}.$$

若 y 依赖 $u_1, \dots, u_m$ ，而每个 u_j 又依赖 x_i ，则所有路径贡献相加：

$$\frac{\partial y}{\partial x_i} = \sum_{j=1}^m \frac{\partial y}{\partial u_j} \frac{\partial u_j}{\partial x_i}.$$

神经网络是复合函数，反向传播就是按计算图反向重复应用这条法则。

2.4.5 小结

导数描述单变量局部变化，偏导描述多变量函数沿单一坐标方向的变化，梯度汇总所有偏导，链式法则把复合函数各层导数连接起来。

2.4.6 练习

原书练习要求绘制函数及其切线，并对给定函数求导，用于验证解析导数的几何含义。

2.5 自动微分

链式法则给出求导规则；自动微分把它落实到程序。前向计算时记录操作组成的计算图，反向阶段逐节点应用链式法则。它计算的是实际程序所表示函数的数值导数，而不是对表达式做符号化简。

2.5.1 一个简单的例子

令

$$y = 2\mathbf{x}^\top \mathbf{x} = 2 \sum_i x_i^2.$$

逐分量求导得到

$$\frac{\partial y}{\partial x_i} = 4x_i, \quad \nabla_{\mathbf{x}} y = 4\mathbf{x}.$$

最小验证代码为：

```
x = torch.arange(4.0, requires_grad=True)
y = 2 * torch.dot(x, x)
y.backward()
x.grad
```

PyTorch 默认累积梯度，因此再次反向传播前通常先清零已有梯度。

2.5.2 非标量变量的反向传播

上例中 y 是标量，可以直接 `backward()`。当 $\mathbf{y} \in \mathbb{R}^m$ 、 $\mathbf{x} \in \mathbb{R}^n$ 时， $\partial \mathbf{y} / \partial \mathbf{x}$ 自然是雅可比矩阵。训练通常不显式构造整个雅可比，而计算向量-雅可比积。若 $\mathbf{y} = \mathbf{x} \odot \mathbf{x}$ ，对输出求和后

$$\nabla_{\mathbf{x}} \sum_i y_i = \nabla_{\mathbf{x}} \sum_i x_i^2 = 2\mathbf{x}.$$

因此 `y.sum().backward()` 等价于用全 1 向量聚合各输出分量的梯度。

2.5.3 分离计算

有时只希望沿部分路径求导。设 $\mathbf{y} = \mathbf{x} \odot \mathbf{x}$ ，令 $\mathbf{u}$ 取与 $\mathbf{y}$ 相同的数值，但通过 `detach()` 与产生 $\mathbf{y}$ 的计算图断开。若

$$\mathbf{z} = \mathbf{u}^\top \mathbf{x},$$

反向传播时 $\mathbf{u}$ 被视为常量，因此

$$\nabla_{\mathbf{x}} \mathbf{z} = \mathbf{u},$$

而不会沿 $\mathbf{u} = \mathbf{x} \odot \mathbf{x}$ 的路径继续求导。

2.5.4 Python 控制流的梯度计算

计算图不必事先固定。PyTorch 使用动态计算图：循环次数和条件分支可以依赖输入；反向传播只沿本次前向实际执行的可微路径进行。若某次执行路径上函数化为 $f(a) = ka$ ，则

$$\frac{df}{da} = k = \frac{f(a)}{a}, \quad a \neq 0.$$

2.5.5 小结

自动微分的最小流程为：给变量启用梯度追踪，执行前向计算，把目标标量反向传播，最后读取梯度。其数学基础是计算图上的链式法则。

2.5.6 练习

原书练习围绕二阶导数开销、重复反向传播、非标量输入和控制流函数展开。

2.5.7 练习解答

2.5.7.1 1. 为什么二阶导数的开销更大？

一阶反向传播本身形成新的导数计算。若继续求二阶导数，必须保留并再次遍历一阶导数的计算图，因此时间和内存开销均增加。

2.5.7.2 2. 连续运行两次反向传播

默认反向传播后计算图会释放，直接对同一图再次反向传播会失败。显式保留计算图可以再次反传，但梯度仍会累积，必须按需要清零。

2.5.7.3 3. 将控制流例子中的输入改成向量或矩阵

输入改为向量或矩阵后，若输出也非标量，需要先求和或提供外部梯度，把反向目标转化为向量-雅可比积。

2.5.7.4 4. 重新设计控制流梯度例子

可用依赖输入符号的条件分支构造分段函数。自动微分对被选中的分支求导，不对离散的分支选择操作求导。

2.6 概率

训练给出点估计之后，还需要用数值表达不确定性。机器学习模型预测的通常不是确定事实，而是给定观测后各结果的可信程度。下面从单事件概率出发，再处理多变量和数字特征。

2.6.1 基本概率论

从分布中抽取结果称为采样。重复独立采样时，事件 $\mathcal{A}$ 的经验频率

$$\hat{P}_N(\mathcal{A}) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{X_i \in \mathcal{A}\}$$

随 N 增大趋近真实概率 $P(\mathcal{A})$ ，这是大数定律在本节中的直观含义。

2.6.1.1 概率论公理

样本空间 $\mathcal{S}$ 包含全部可能结果，事件是 $\mathcal{S}$ 的子集。概率满足

$$\begin{aligned} P(\mathcal{A}) &\geq 0, \\ P(\mathcal{S}) &= 1, \\ P\left(\bigcup_{i=1}^{\infty} \mathcal{A}_i\right) &= \sum_{i=1}^{\infty} P(\mathcal{A}_i) \quad (\mathcal{A}_i \text{ 两两互斥}). \end{aligned}$$

由此可得 $P(\emptyset) = 0$ 。

2.6.1.2 随机变量

随机变量 X 把样本空间中的结果映射到数值。离散变量可直接讨论 $P(X = x)$ ；连续变量用密度描述，单点概率可以为 0，而区间概率仍可为正。

2.6.2 处理多个随机变量

单个随机变量还不够：疾病与检测、图像与标签都成对出现。多个变量之间的关系由联合分布描述，条件化和边际化分别表示“已知某变量”与“消去某变量”。

2.6.2.1 联合概率

联合概率 $P(A = a, B = b)$ 表示两个条件同时成立。因为联合事件包含于任一边际事件，

$$P(A = a, B = b) \leq P(A = a).$$

2.6.2.2 条件概率

联合概率给出同时发生的程度。当 $P(A = a) > 0$ 时，条件概率是在已知 $A = a$ 的子空间内重新归一化：

$$P(B = b | A = a) = \frac{P(A = a, B = b)}{P(A = a)}.$$

2.6.2.3 贝叶斯定理

把条件概率的定义与乘法法则合在一起，就可以交换条件方向。由

$$P(A, B) = P(B | A)P(A) = P(A | B)P(B)$$

得到

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}, \quad P(B) > 0.$$

它把先验 $P(A)$ 与似然 $P(B | A)$ 合成为后验 $P(A | B)$ 。

2.6.2.4 边际化

贝叶斯公式的分母 $P(B)$ 往往未知，需要对未观测变量求和得到。若 A 为离散变量，

$$P(B) = \sum_A P(A, B).$$

连续变量对应的操作是积分。

2.6.2.5 独立性

若

$$P(A, B) = P(A)P(B),$$

则记为 $A \perp B$ 。给定 C 后的条件独立定义为

$$A \perp B | C \iff P(A, B | C) = P(A | C)P(B | C).$$

2.6.2.6 应用

原书以 HIV 检测说明先验的重要性。若感染率 $P(H = 1) = 0.0015$ ，且第一次检测满足

$$P(D_1 = 1 | H = 0) = 0.01, \quad P(D_1 = 1 | H = 1) = 1,$$

则

$$P(D_1 = 1) = 0.01(1 - 0.0015) + 1(0.0015) = 0.011485,$$

因此一次阳性后的感染概率为

$$P(H = 1 \mid D_1 = 1) = \frac{1 \times 0.0015}{0.011485} \approx 0.1306.$$

结论是：即使检测灵敏度高，低先验事件的一次阳性也不等于高后验概率。

2.6.3 期望和方差

概率描述事件；期望和方差把分布压缩成数字特征。离散随机变量的期望为

$$\mathbb{E}[X] = \sum_x xP(X = x).$$

对函数 f ,

$$\mathbb{E}_{x \sim P}[f(x)] = \sum_x f(x)P(x).$$

方差衡量变量围绕期望的平方偏离：

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2.$$

标准差为方差平方根，与原变量具有相同量纲。

2.6.4 小结

概率从样本空间和事件出发，用联合、条件、边际和独立性处理多变量关系，再用期望和方差概括分布的位置与离散程度。

2.6.5 练习

原书练习要求分析抽样频率、条件概率、贝叶斯更新、独立性以及期望和方差。

2.7 查阅文档

前面各节用到的接口会随框架版本变化，因此应掌握发现和核验方法，而不是记忆完整函数表。先列出模块内容，再查看单个对象的用法。

2.7.1 查找模块中的所有函数和类

`dir(module)` 列出模块中的属性、函数和类。双下划线包围的名称通常属于 Python 特殊机制，单下划线开头的名称通常是内部实现。

2.7.2 查找特定函数和类的用法

从列表选定对象后, `help(object)` 显示文档字符串; Jupyter 中的 `?` 查看帮助, `??` 在可用时显示 Python 源码。最终应以当前安装版本的官方文档为准。

2.7.3 小结

查阅接口的顺序是: 先用 `dir` 发现候选对象, 再用 `help` 或 Jupyter 帮助查看参数、返回值与示例。

2.7.4 练习

原书练习要求查询张量函数的官方文档并运行示例, 以建立独立核验 API 的习惯。